

XM · CLASE 3



Soluciones de Analítica con Databricks

Clase 3: Unity Catalog: gobierno, seguridad y acceso

Docente: Manuela Larrea Gómez

Agenda de la clase 3



Quién puede ver qué, escrito en el catálogo y no en un correo

01

40 MIN

Teoría: Unity Catalog de punta a punta

Modelo de objetos · privilegios y herencia · ambientes y grupos · filtros de filas y máscaras · tags · linaje y auditoría · secretos · Delta Sharing

02

10 MIN

Demo admin: catálogos dev / qa / prod

Lo que solo el administrador puede hacer: catálogos, esquemas por capa, grupos por rol, grants de la matriz

03

60 MIN

Lab 3: gobernar tu propio esquema

GRANT y REVOKE con un compañero · PERMISSION_DENIED a propósito · filtro de filas y máscara · tags · linaje · docs/governance.md

04

10 MIN

Cierre

Tag s03 · así sería en Azure · próxima clase: ingesta con Auto Loader

GOBIERNO

Unity Catalog

Un catálogo que sabe qué existe, quién es dueño, quién puede leer y quién leyó



El modelo de objetos: metastore → catálogo → esquema → objeto

Los permisos se heredan hacia abajo. Modelos y funciones están al mismo nivel que las tablas

Metastore

Uno por región. Contiene todos los catálogos y define quién puede crearlos

Catálogo

dev · qa · prod (hoy: workspace). Separa ambientes o grandes dominios

Esquema

bronze_energia · silver_energia · gold_energia · models · c01_<usuario>

Objetos

tablas · vistas · volúmenes · funciones · modelos

```
catálogo . esquema . objeto    dev.silver_energia.demanda_diaria  ·  dev.models.demanda_7d  ·  
/Volumes/dev/bronze_energia/raw_demanda/
```

GRANT sobre un esquema aplica a todas sus tablas, incluidas las que se creen mañana. GRANT sobre una tabla aplica solo a esa.

Seis tipos de objeto, y para qué los usamos en el curso



Tablas

Bronce, plata y oro. Delta, con historial

Clases 1, 4-7



Vistas

Exponer una tabla con filtros o columnas elegidas, sin copiarla

Hoy



Volúmenes

Archivos: la zona de aterrizaje de bronce

Clases 1, 4



Funciones

Filtros de filas y máscaras son funciones SQL registradas

Hoy



Modelos

El registro de MLflow vive en el catálogo, con owner y permisos

Clase 10



Credenciales y ubicaciones externas

Conectan el catálogo con ADLS. No existen en Free Edition

Azure

Todo objeto tiene un owner. Solo el owner o un admin puede dar permisos sobre él.

Tres principios que se exigen desde hoy

Los vas a ver aplicados en la demo, en el lab y en docs/governance.md



Permisos a grupos, nunca a personas

Si alguien se va, se le saca del grupo; no se buscan sus permisos en cuarenta tablas.

Grupos por rol: grp_ingenieria,
grp_analitica, grp_negocio.

```
GRANT SELECT ON SCHEMA prod.gold_energia  
TO `grp_negocio`
```



Mínimo privilegio

Quien lee no escribe. Quien escribe en dev no escribe en prod. El pipeline escribe con su propia identidad (service principal), no con la de una persona.

En qa y prod, ninguna persona tiene
MODIFY



El dueño decide

Cada esquema y tabla tienen owner. Hoy tú eres owner de c01_<usuario>; por eso puedes practicar grants sin el admin.

```
ALTER SCHEMA ... OWNER TO `grp_ingenieria`
```

Los siete privilegios que cubren el 90 % de los casos

Sintaxis SQL estándar: GRANT, REVOKE, SHOW GRANTS. Se versiona en el repo como cualquier código

Privilegio	Sobre	Permite
USE CATALOG	catálogo	Entrar. Sin esto no se ve nada, aunque tengas SELECT
USE SCHEMA	esquema	Entrar al esquema. Segundo pase obligatorio
SELECT	tabla, vista, esquema	Leer
MODIFY	tabla, esquema	Insertar, actualizar, borrar
CREATE TABLE / VOLUME / FUNCTION	esquema	Crear objetos
READ VOLUME / WRITE VOLUME	volumen	Leer o escribir archivos
EXECUTE	función, modelo	Ejecutar una función o invocar un modelo

```
grants.sql
-- pase de entrada
GRANT USE CATALOG ON CATALOG dev
  TO `grp_analitica`;
GRANT USE SCHEMA, SELECT
  ON SCHEMA dev.silver_energia
  TO `grp_analitica`;

-- quitar
REVOKE SELECT ON TABLE
  workspace.c01_ana.demanda_raw
  FROM `account users`;

-- comprobar
SHOW GRANTS ON SCHEMA dev.silver_energia;
```

ALL PRIVILEGES existe. Fuera de un sandbox personal, es la tentación que hay que evitar.

Ambientes por catálogo, roles por grupo: la matriz que cabe en una diapositiva

	dev.bronze / silver	dev.gold	qa.* / prod.bronze, silver	prod.gold	*.models
grp_ingenieria	USE, SELECT, MODIFY, CREATE	SELECT	SELECT	SELECT	SELECT
grp_analitica	USE, SELECT	SELECT, MODIFY, CREATE	SELECT	SELECT	SELECT, EXECUTE, CREATE MODEL (dev)
grp_negocio	—	—	—	SELECT (vía vistas)	—
sp_pipeline (clase 13)	USE, SELECT, MODIFY	USE, SELECT, MODIFY	USE, SELECT, MODIFY	USE, SELECT, MODIFY	EXECUTE, MODIFY

Si la matriz no cabe en una diapositiva, el diseño está mal. Se lee igual en Free Edition y en Azure; allá los grupos vienen de Entra ID.

Sandbox: cada persona es owner de dev.sb_<usuario> (hoy: workspace.c01_<usuario>). Nadie escribe a mano en qa ni en prod.

Filtros de filas y máscaras de columnas: la misma tabla, cada quien ve lo suyo



Filtro de filas

Una función que devuelve TRUE/FALSE por fila. Se asocia a la tabla y se evalúa según quién consulta. Sin copias de la tabla.



Máscara de columna

Una función que transforma el valor: el real para un grupo, redondeado o nulo para el resto.



En XM

Un operador de red ve solo sus series; negocio ve la demanda en miles de kWh; auditoría ve todo. Una tabla, tres vistas del mundo.

```
row_filter_mask.sql
```

```
CREATE FUNCTION solo_regulado(tipo STRING)
RETURNS BOOLEAN
RETURN is_account_group_member('grp_negocio')
      = FALSE OR tipo = 'Regulado';
```

```
CREATE FUNCTION mascara_kwh(v DOUBLE)
RETURNS DOUBLE
RETURN CASE WHEN is_account_group_member(
      'grp_negocio') THEN round(v, -3) ELSE v END;
```

```
ALTER TABLE demanda_raw
  SET ROW FILTER solo_regulado ON (TipoMercado);
ALTER TABLE demanda_raw ALTER COLUMN Valor
  SET MASK mascara_kwh;
```

`is_account_group_member()` es la clave: la lógica vive en el catálogo, no en cada notebook.

Tags, comentarios y clasificación: los contratos, dentro del catálogo

Lo que escribiste en docs/architecture.md, visible en Catalog Explorer sin abrir el repo



Comentarios

En tabla y en columna. El explorador los muestra; el asistente los usa para entender el dato.

```
COMMENT ON TABLE demanda_raw IS 'Demanda y pérdidas por serie-día, tal como publica XM';
```



Tags

Pares clave-valor. Los del curso: capa, dominio, owner, sensibilidad.

```
ALTER TABLE demanda_raw SET TAGS ('capa'='bronze',  
'dominio'='energia', 'owner'='grp_ingenieria',  
'sensibilidad'='interna');
```



Clasificación por tag

Una política escrita una vez: toda columna con sensibilidad = confidencial se enmascara. Se consulta en information_schema.column_tags.

```
SELECT * FROM system.information_schema.table_tags WHERE  
schema_name = 'c01_ana';
```

Linaje y auditoría: dos cosas que XM va a pedir sin que las hayan pedido



Linaje automático

Unity Catalog lo captura en tiempo de ejecución, tabla a tabla y columna a columna. Una vista sobre tu tabla ya dibuja la flecha en Catalog Explorer → pestaña Lineage. Responde: ¿de dónde sale este número? ¿qué se rompe si cambio esta columna?



Auditoría en tablas de sistema

Cada acceso queda en system.access.audit: quién, qué objeto, cuándo, desde dónde. Y system.information_schema es el catálogo consultable con SQL: privilegios, tags, filtros, máscaras. La celda verificar() de hoy lee de ahí.

```
auditoria.sql
-- ¿quién tiene qué sobre mi tabla?
SELECT grantee, privilege_type FROM system.information_schema.table_privileges
WHERE table_schema = 'c01_ana' AND table_name = 'demanda_raw';
-- ¿quién leyó mi tabla esta semana?
SELECT user_identity.email, event_time FROM system.access.audit WHERE request_params.full_name_arg LIKE '%c01_ana%';
```

Secretos y credenciales: nada de contraseñas en notebooks



Secret scopes

Almacén de claves. El código lee con `dbutils.secrets.get(scope, key)`; el valor nunca aparece en pantalla ni en logs (se muestra como [REDACTED]). Free Edition: scope gestionado por Databricks. Azure: scope respaldado por Key Vault.



Storage credentials y external locations

Los objetos que conectan el catálogo con el almacenamiento (ADLS Gen2). Quién puede crear una tabla sobre un contenedor es un permiso más de Unity Catalog. No existen en Free Edition.

```
secretos.py
# nunca así
token = "dapi1234..."
# así
token = dbutils.secrets.get(scope="xm-demanda", key="simem-api-key")
```

El repo tiene un check de pre-commit (gitLeaks) que bloquea tokens antes de que lleguen a GitHub.

Compartir sin copiar: Delta Sharing y Clean Rooms

Solo teoría hoy; son la respuesta cuando un agente del mercado pida "los datos"



Delta Sharing

Protocolo abierto para compartir tablas con otra organización. El receptor lee en vivo con cualquier herramienta (Spark, pandas, Power BI) sin recibir archivos. Se revoca cuando se quiera; el auditor ve cada lectura. Caso XM: entregar a un operador de red solo sus series, actualizadas a diario.



Clean Rooms

Dos partes aportan datos para un cálculo conjunto sin que ninguna vea los datos crudos de la otra. Caso XM: cruzar demanda con datos de un comercializador para un estudio sin que ninguno entregue su base al otro.

Ambas se apoyan en Unity Catalog: lo que se comparte es un objeto gobernado, con owner, permisos y auditoría.

Sector: por qué el gobierno de datos pesa más en el mercado eléctrico

Información comercial de agentes que compiten entre sí, bajo un regulador que exige trazabilidad



Confidencialidad por agente

Lo que un comercializador vende, a quién y cuánto es sensible frente a sus competidores. La CREG define qué se publica agregado y qué no.



Trazabilidad exigible

Regulador, Superintendencia y auditorías piden quién vio qué y cuándo. `system.access.audit` es la respuesta técnica.



Referencia: ENTSO-E Transparency Platform

Los operadores europeos publican generación, carga y precios en formato común, con catálogo de qué se publica y con qué retraso. Gobierno de datos hecho norma.

Lo de hoy es la versión interna: antes de publicar afuera, saber adentro quién ve qué.

Free Edition vs. Azure Databricks: qué simulamos hoy

Elemento	Hoy en Free Edition	En XM sobre Azure
Ambientes	Tres catálogos en un workspace (dev, qa, prod)	Tres workspaces; cada uno con su catálogo por defecto; catálogos compartidos por binding
Quién crea catálogos	La docente, admin del workspace y del metastore	Rol de plataforma (admin de cuenta), no el equipo de analítica
Grupos	Creados a mano en Identity and access	Grupos de Entra ID sincronizados por SCIM
Identidad del pipeline	Un usuario (hasta la clase 13)	Service principal por ambiente
Secretos	Secret scope de Databricks	Secret scope respaldado por Azure Key Vault
Almacenamiento	Gestionado por Databricks	ADLS Gen2 con storage credential + external location
Auditoría	system.access.audit	Lo mismo, exportado a Log Analytics / SIEM

El SQL de grants, filtros y máscaras es idéntico.

Demo admin: lo que solo el administrador puede hacer

clases/c03-unity-catalog/admin_setup_gobierno.sql · 10 min en vivo

1

Catálogos

CREATE CATALOG dev, qa, prod. Requiere privilegio sobre el metastore: en XM es un rol de plataforma

2

Esquemas por capa

En dev: bronze_energia, silver_energia, gold_energia, models. En qa y prod, lo mismo (vacíos hasta la clase 13)

3

Grupos por rol

grp_ingenieria, grp_analitica, grp_negocio ya creados; cada estudiante está en uno

4

Grants de la matriz

Fila por fila, tal como la diapositiva 8. SHOW GRANTS para comprobar

admin_setup_gobierno.sql · sección 3

```
CREATE CATALOG IF NOT EXISTS dev;
CREATE SCHEMA IF NOT EXISTS dev.bronze_energia;
GRANT USE CATALOG ON CATALOG dev
    TO `grp_ingenieria`, `grp_analitica`;
GRANT USE SCHEMA, SELECT, MODIFY, CREATE TABLE
    ON SCHEMA dev.bronze_energia
    TO `grp_ingenieria`;
```

```
-- un estudiante de grp_negocio intenta:
CREATE TABLE dev.bronze_energia.prueba (x INT);
-- PERMISSION_DENIED ← el gobierno funciona
```

Ese PERMISSION_DENIED en vivo es la demostración. Léalo completo: dice qué privilegio falta y sobre qué objeto.

Lab 3 (1/2): gobernar tu propio esquema, con un compañero

clases/c03-unity-catalog/notebooks/lab03_gobierno · widget usuario y widget companero

1 GRANT a account users y comprobar

GRANT USE SCHEMA, SELECT ... TO `account users`; SHOW GRANTS

2 El compañero consulta tu tabla

Funciona. Anota cuántas filas ve

3 REVOKE y el compañero vuelve a consultar

PERMISSION_DENIED. Léanlo: privilegio y objeto

4 Filtro de filas y máscara

Dos funciones + ALTER TABLE ... SET ROW FILTER / SET MASK

5 GRANT otra vez; el compañero consulta

Ve solo Regulado y Valor redondeado. Tú, owner, ves todo

```
lab03 · pasos 1-3

-- tú (owner)
GRANT USE SCHEMA ON SCHEMA
  workspace.c01_ana TO `account users`;
GRANT SELECT ON TABLE
  workspace.c01_ana.demanda_raw
  TO `account users`;

-- compañero
SELECT count(*) FROM
  workspace.c01_ana.demanda_raw;

-- tú
REVOKE SELECT ON TABLE
  workspace.c01_ana.demanda_raw
  FROM `account users`;
```

Error típico: SELECT en la tabla sin USE SCHEMA en el esquema. Los dos hacen falta.

Lab 3 (2/2): tags, linaje, governance.md y verificación

Ya solo · rama feature/c03-gobierno

6

Tags y comentarios

SET TAGS capa, dominio, owner, sensibilidad; COMMENT ON TABLE y en dos columnas

7

Vista y linaje

CREATE VIEW demanda_regulado_v ... ; Catalog Explorer → Lineage

8

docs/governance.md

Tu matriz: catálogos, esquemas, grupos, privilegios; secretos; auditoría

9

verificar()

Lee information_schema: grants, tags, filtro, máscara; y el documento

10

Commit, push, PR

feat(c03): gobierno del esquema y governance.md

Criterios de aceptación

- Tabla con filtro de filas y máscara activos
- 4 tags y comentario de tabla
- Vista demanda_regulado_v con linaje visible
- governance.md sin marcadores <...>
- verificar() en OK; PR en verde

Comparar con la solución

Al cierre se libera s03. En GitHub:
[github.com/<docente>/Databricks-to-XM/compare/s03...<tú>:Databricks-to-XM:main](https://github.com/Databricks-to-XM/compare/s03...<tú>:Databricks-to-XM:main) · o en terminal: git diff s03 -- docs/governance.md

Así sería en Azure Databricks: gobierno con Entra ID, Key Vault y ADLS

Hoy	En XM
Tres catálogos en un workspace	Un workspace por ambiente; el catálogo prod solo está enlazado (binding) al workspace de prod
Grupos creados a mano	Grupos de Entra ID (p. ej. XM-Analitica-Energia) sincronizados con SCIM; el owner de cada esquema es un grupo
Filtro y máscara con <code>is_account_group_member</code>	Idéntico
Secret scope de Databricks	Scope respaldado por Key Vault: la clave vive en Key Vault, Databricks solo la lee
Almacenamiento gestionado	ADLS Gen2: storage credential (managed identity) + external location por capa
<code>system.access.audit</code>	Exportado a Log Analytics; alertas de acceso inusual

Lo que no cambia: la matriz de `governance.md`, los GRANT, las funciones de filtro y máscara.

Resumen de la clase 3 y preparación para la clase 4

Logros de hoy

- ✓ **Modelo de objetos y herencia**
metastore → catálogo → esquema → objeto
- ✓ **Permisos a grupos, mínimo privilegio, owner**
La matriz de dev/qa/prod por rol
- ✓ **Filtros de filas y máscaras**
Una tabla, cada quien ve lo suyo
- ✓ **Tags, linaje, auditoría, secretos**
Contratos dentro del catálogo
- ✓ **PERMISSION_DENIED a propósito**
Hito 2 abierto

Próxima clase

- ➔ **Ingesta con Auto Loader**
Del upload manual a un proceso que vigila el volumen
- ➔ **La primera republicación**
Llega un archivo con días que ya existen: bronce acumula
- ➔ **Idempotencia y metadatos**
`_ingested_at`, `_source_file`, `_publication_date`
- ➔ **Traer a la clase 4**
Lab 3 terminado y governance.md en tu fork



Gracias

Nos vemos en la clase 4: Ingesta y repúblicas

DOCENTE

Manuela Larrea Gómez